

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Лабораторная работа №2
по дисциплине «Основы программной инженерии»
Вариант: 1232

Преподаватель:
Лазеев Сергей Максимович

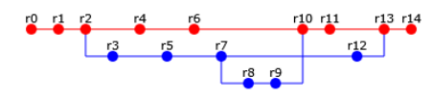
Выполнил:
Караганов Павел Эдуардович
Группа: Р3210

Содержание

Блок-схема варианта	3
1 Список команд, использованных для создания и настройки Git-репозитория	3
2 Ревизии Git и последовательности команд	3
3 Скрипт для создания	4
4 Выводы	6

Блок-схема варианта

Вариант



Сконфигурировать в своём домашнем каталоге репозитории `svn` и `git` и загрузить в них начальную ревизию файлов с исходными кодами (в соответствии с выданным вариантом).

Воспроизвести последовательность команд для систем контроля версий `svn` и `git`, осуществляющих операции над исходным кодом, приведённые на блок-схеме.

При составлении последовательности команд необходимо учитывать следующие условия:

- Цвет элементов схемы указывает на пользователя, совершившего действие (красный - первый, синий - второй).
- Цифры над узлами - номер ревизии. Ревизии создаются последовательно.
- Необходимо разрешать конфликты между версиями, если они возникают.

1 Список команд, использованных для создания и настройки Git-репозитория

1. `bash remove_git.bash` – удаление ранее созданного каталога репозитория перед повторным запуском.
2. `mkdir repo_git` – создание каталога локального Git-репозитория.
3. `cd repo_git` – переход в рабочий каталог репозитория.
4. `git init` – инициализация пустого Git-репозитория.
5. `unzip -o ../commits/commitN.zip -d .` – восстановление состояния файлов для ревизии `rN`.
6. `git add .` – добавление изменений в индекс.
7. `git commit -m "rN" --author="..."` – создание коммита ревизии `rN` с нужным автором.
8. `git checkout -b blue_branch_1` – создание и переключение на ветку `blue_branch_1`.
9. `git checkout -b blue_branch_2` – создание и переключение на ветку `blue_branch_2`.
10. `git checkout main` – возврат в основную ветку `main`.
11. `git merge --no-ff --no-commit <branch>` – слияние ветки без авто-коммита (для ручной обработки конфликтов).

2 Ревизии Git и последовательности команд

Ревизия	Последовательность команд	Комментарий
r0	<code>unzip commit0.zip -> git add . -> git commit -m "r0" --author="Red ..."</code>	Начальное состояние файлов в основной ветке.
r1	<code>unzip commit1.zip -> git add . -> git commit -m "r1" --author="Red ..."</code>	Последовательное изменение в <code>main</code> .
r2	<code>unzip commit2.zip -> git add . -> git commit -m "r2" --author="Red ..."</code>	Последний коммит до первого ветвления.
r3	<code>git checkout -b blue_branch_1 -> unzip commit3.zip -> git commit ...</code>	Первый коммит синего автора в ветке <code>blue_branch_1</code> .
r4	<code>git checkout main -> unzip commit4.zip -> git commit ...</code>	Продолжение разработки в основной ветке.
r5	<code>git checkout blue_branch_1 -> unzip commit5.zip -> git commit ...</code>	Изменение в параллельной ветке <code>blue_branch_1</code> .

r6	git checkout main -> unzip commit6.zip -> git commit ...	Следующая ревизия в main.
r7	git checkout blue_branch_1 -> unzip commit7.zip -> git commit ...	Подготовка к выделению дополнительной ветки.
r8	git checkout -b blue_branch_2 -> unzip commit8.zip -> git commit ...	Создана ветка blue_branch_2 от blue_branch_1.
r9	unzip commit9.zip -> git add . -> git commit -m "r9" --author="Blue ..."	Развитие ветки blue_branch_2.
r10	git checkout main -> git merge --no-ff --no-commit blue_branch_2 -> разрешение конфликтов -> unzip commit10.zip -> git commit ...	Фиксация результата первого слияния в main.
r11	unzip commit11.zip -> git add . -> git commit -m "r11" --author="Red ..."	Обычный коммит в main после merge.
r12	git checkout blue_branch_1 -> unzip commit12.zip -> git commit ...	Дополнительное изменение в blue_branch_1.
r13	git checkout main -> git merge --no-ff --no-commit blue_branch_1 -> разрешение конфликтов -> unzip commit13.zip -> git commit ...	Фиксация результата второго слияния в main.
r14	unzip commit14.zip -> git add . -> git commit -m "r14" --author="Red ..."	Завершающая ревизия в основной ветке.

3 Скрипт для создания

```
#!/bin/bash
```

```
RED="Red <red@example.com>"
BLUE="Blue <blue@example.com>"
```

```
REPO_DIR="repo_git"
COMMITTS_DIR="../commits"
```

```
bash remove_git.bash
mkdir "$REPO_DIR"
cd "$REPO_DIR" || exit
```

```
git init
```

```
make_commit() {
    local rev=$1
    local author=$2

    unzip -o "${COMMITTS_DIR}/commit${rev}.zip" -d .

    git add .

    git commit -m "r${rev}" --author="$author"
```

```

    echo "Коммит r${rev} создан."
}

make_commit 0 "$RED"
make_commit 1 "$RED"
make_commit 2 "$RED"

git checkout -b blue_branch_1

make_commit 3 "$BLUE"

git checkout main
make_commit 4 "$RED"

git checkout blue_branch_1
make_commit 5 "$BLUE"

git checkout main
make_commit 6 "$RED"

git checkout blue_branch_1
make_commit 7 "$BLUE"

git checkout -b blue_branch_2

make_commit 8 "$BLUE"
make_commit 9 "$BLUE"

git checkout main
#git merge --no-ff blue_branch_2 --no-commit -Xours(theirs)

git merge --no-ff --no-commit blue_branch_2
if [ $? -ne 0 ]; then
    echo "Конфликт! Разрешите его и нажмите Enter, чтобы продолжить..."
    read -p "Нажмите Enter для продолжения..."
fi

make_commit 10 "$RED"
make_commit 11 "$RED"

git checkout blue_branch_1
make_commit 12 "$BLUE"

git checkout main

#git merge --no-ff blue_branch_1 --no-commit -Xours(theirs)
git merge --no-ff --no-commit blue_branch_1
if [ $? -ne 0 ]; then
    echo "Конфликт! Разрешите его и нажмите Enter, чтобы продолжить..."
    read -p "Нажмите Enter для продолжения..."
fi

make_commit 13 "$RED"

```

```
make_commit 14 "$RED"
```

4 Выводы

В ходе работы была воспроизведена заданная история изменений для Git-репозитория с последовательными коммитами, ветвлениями и слияниями. При выполнении merge использовался режим без автоматического коммита (--no-commit), что позволило контролируемо разрешать конфликты и фиксировать итоговое состояние в отдельных ревизиях. Полученная структура истории соответствует блок-схеме варианта и подтверждает корректность построенной последовательности действий в Git.